Chapter 7: Using OLE DB Templates for Data Access

OLE DB is a set of COM interfaces that provide uniform access to data stored in diverse data sources. Using OLE DB, your application can access data stored in DBMS (database management system) and non-DBMS data sources.

DBMS data sources are traditional databases that store data in tabular format. Non-DBMS data sources are data containers that do not store data in tabular format. Some common examples of non-DBMS data sources are electronic mail boxes, text files, and spreadsheets.

In this chapter you will learn to use OLE DB consumer templates to build applications that access data stored in DBMS and non-DBMS data sources.

Objectives

After completing this chapter, you will be able to:

- List the benefits of using the OLE DB technology for data access.
- Describe the role of OLE DB components in data access.
- Explain the architecture of provider and consumer template classes.
- Build consumer applications by using OLE DB consumer templates.

212 Overview of OLE DB

This section discusses the need for a generic data access interface standard such as OLE DB and briefly describes the different types of OLE DB applications and OLE DB components.

This section includes the following topics:

- Benefits of OLE DB
- Types of OLE DB Applications
- OLE DB Components

Notes

213 Benefits of OLE DB

Open Database Connectivity (ODBC), which is the industry standard for data access interfaces, requires data to be stored in a tabular format. However, a vast amount of critical information necessary to conduct day-to-day business is found outside traditional databases. To take advantage of the benefits of database technology, such as queries, transactions, and security, organizations were compelled to move the data from its original containing system into a DBMS.

Over time, businesses realized that transferring data into a DBMS is an expensive and redundant operation. This realization led to the development of OLE DB, which is a more generalized and more efficient strategy for data access than ODBC.

To see an expert point of view about the need for a universal data access technique and the role of OLE DB in data access, click this icon.

 (CD-ROM plays the expert point-of-view, "Universal Data Access.")

Some benefits of using OLE DB include:

- Access to component DBMSs

  Often, applications do not require or use all the functionality packaged in a commercial, monolithic DBMS. Yet, they are forced to pay additional resource overhead for functionality they do not need. OLE DB divides a DBMS into functional areas, such as data access and updates (rowsets), query processing, catalog information, notifications, transactions, security, and remote data access. By defining a DBMS as a set of well-defined components, OLE DB enables applications to use only the necessary DBMS functionality.

- Open, extensible collection of interfaces

  OLE DB interfaces define the boundaries of DBMS components, such as row containers and query processors, to enable uniform access to diverse information sources. This approach views a DBMS as a conglomerate of cooperating components that consume and produce data through a uniform set of interfaces.

- Support for diverse information sources

  OLE DB makes it easy for applications to access data stored in diverse DBMS and non-DBMS information sources. DBMS sources may include mainframe databases such as IMS and DB2, server databases such as Oracle and Microsoft SQL Server, and desktop databases such as Microsoft Access, Paradox, and Microsoft FoxPro®. Non-DBMS sources can include information stored in file systems such as Windows NT and UNIX, indexed sequential files, e-mail, spreadsheets, project management tools, and many other sources.

Notes

214

Types of OLE DB Applications

The two types of OLE DB applications are consumers and providers.

Consumers

A consumer application retrieves data from a data source for further processing. Examples of consumers are Visual C++-based and Visual Basic-based applications that access an Oracle or Access database.

Providers

There are two types of provider applications:

- Data providers

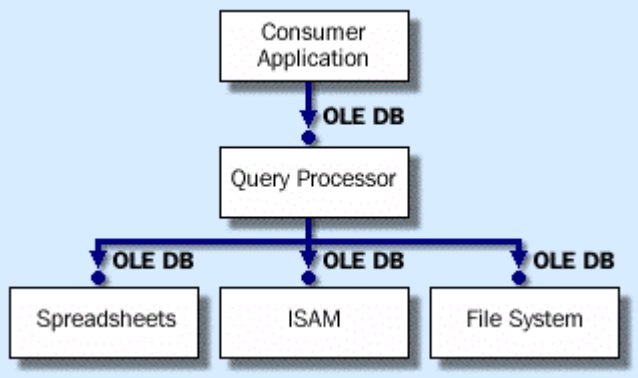
  A data provider is any application that owns data. It can expose its data in a tabular format, regardless of the format in which it stores data. Examples of data providers are relational DBMSs, spreadsheets, indexed sequential access methods (ISAMs), and e-mail.

- Service providers

  A service provider does not have its own data, but it facilitates data access between a data provider and a consumer. Examples of service providers are query processors and cursor engines. Service providers act both as consumers and providers. In its role as a consumer, a service provider retrieves data from the base tables owned by the data provider. In its role as a provider, the query processor creates a rowset from the retrieved data and returns it to the consumer.

For more information about OLE DB providers, search for "Using the OLE DB Provider Templates" in the MSDN Library Visual Studio 6.0.

The following illustration shows the relationship between the consumer, data provider, and service provider.

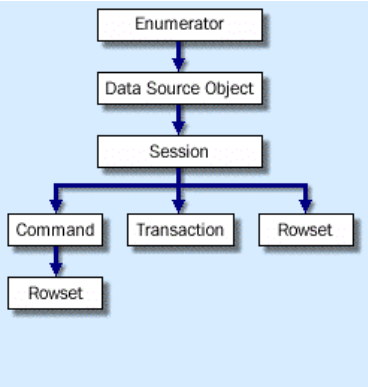


Notes

215 OLE DB Components

OLE DB consists of several components, each of which is a COM object. To see an illustration of OLE DB components, click this icon.

(CD-ROM displays the following illustration.)



To see an animation that describes the OLE DB object model, click this icon.

(CD-ROM plays the animation, "OLE DB Object Model.")

Enumerators

An enumerator is a COM object that can locate available data sources and other enumerators. Consumers that are not customized for a particular data source use enumerators to find a data source to use.

The OLE DB Software Development Kit (SDK) includes a root enumerator that traverses the registry looking for data sources and other enumerators. Other enumerators traverse the registry or search in a provider-specific manner. For example, the MSDASQL enumerator will search for ODBC data sources.

Data Source Objects

Data source objects contain the code to connect to a data source, such as a file or a DBMS. To access an OLE DB provider, a consumer must first create an instance of a data source object. The data source object exposes IDBInitialize, which the consumer uses to connect to the data source. Once a data source object has been successfully initialized, the consumer can call methods in IDBProperties to query the capabilities of a provider. These capabilities include rowset properties such as scrollability and transaction properties such as supported isolation levels.

Notes

Sessions

A data source object creates one or more sessions. The primary function of a session is to provide a context for transactions. A session can be in one of the following modes:

- Manual commit

  If you call the **CSession** member function **StartTransaction** to start an explicit transaction, the session is in manual commit mode. You must explicitly commit or abort any work done in the session. You can call **StartTransaction** only after the session has been successfully created.

- Auto-commit

  If transactions are not supported or if the call to **StartTransaction** is not made, the session is in auto-commit mode. Any work done in the session is automatically committed; it cannot be aborted.

Transactions

A transaction enables consumers to define units of work within a provider. These units of work have the atomicity, concurrency, isolation, and durability (ACID) properties. A transaction enables more flexible access to data among concurrent consumers.

For providers that support nested transactions, calling **StartTransaction** within an existing transaction begins a new nested transaction below the current transaction. Calling the **Commit** or **Abort** method commits or aborts the transaction at the lowest level, respectively.

Commands

In OLE DB, data definition language (DDL) and data manipulation language (DML) statements are referred to as text commands. A command object contains a text command and encapsulates the query processing services available in DBMSs.

Executing a command such as an SQL **SELECT** statement creates a rowset, while executing a command such as an SQL **UPDATE** or **CREATE TABLE** statement does not create a rowset.

Text commands are expressed in a provider-specific language, which is typically ANSI SQL92.

Rowsets

The rowset object enables OLE DB data providers to expose data in tabular form. A rowset is a set of rows in which each row has columns of data. Base table providers present their data in the form of rowsets. Query processors present the result of queries in the form of rowsets.

Rowsets can be created in one of the following ways:

- As a result of a query

  This method is supported by all providers.

- By calling **IOpenRowset::OpenRowset**

  Simple providers, such as those built over a base table, index, file, or in-memory structure, generally do not support this method.

Notes

Errors

Each OLE DB method provides a return code that indicates the success or failure of the method. There are two types of return codes.

- Success or warning

  Success or warning codes begin with S_ or DB_S_ and indicate that the method successfully completed.

- Error

  Error codes begin with E_ or DB_E_ and indicate that the method failed completely and was unable to do any useful work.

Notes

216 Introduction to OLE DB Templates

OLE DB Templates refers to a set of classes that help you develop OLE DB applications with minimum effort. You use these classes to implement many of the commonly used OLE DB object interfaces.

The OLE DB templates are provided in the form of a template library that is similar to the Active Template Library (ATL). The classes in this template library can be grouped into:

- Provider classes

  You use these classes to implement a database provider application.
- Consumer classes

  You use these classes to implement a database consumer application.

This section describes the provider and consumer templates and their architecture.

This section includes the following topics:

- OLE DB Provider Templates
- OLE DB Consumer Templates

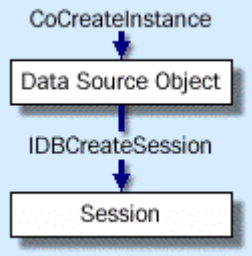
Notes

217 OLE DB Provider Templates

The OLE DB Provider Templates support the OLE DB version 1.1 specification. Using these templates, you can implement simple read-only provider applications. You can also implement read/write providers using these templates, but you need to add code manually. The provider applications may or may not support commands.

For example, you can build a provider that executes the DOS **DIR** command to query the file system. The provider will return the directory information in a rowset, which is the standard OLE DB mechanism for returning tabular data.

The OLE DB provider architecture includes a data source object and one or more session objects, as shown in the following illustration.



When a consumer application needs data, it calls the **CoCreateInstance** function to create a data source object, as shown in the following example:

```
int main()
{
   // Initialize OLE
   ...

   // Create an instance of a data source object for an email data provider.
   CoCreateInstance(CLSID_MailProvider, 0, CLSCTX_LOCAL_SERVER,
IID_IDBInitialize, (void**) &pIDBInit);
   // Initialize the data source object for email data provider.
```

Each data provider is identified by a unique class identifier (CLSID) in the registry. The **CoCreateInstance** function creates an instance of the data source object through the object's class factory.

When a data source object is created, it is in an uninitialized state. For example, if the data source is a file, the data source object has not yet opened the file; if the data source is an SQL database, the data source object has not yet connected to the database.

The consumer can initialize the data source object by calling the **IDBInitialize::Initialize** function exposed by the data source object. After a data source object has been successfully initialized, the consumer calls the **IDBCreateSession::CreateSession** function to create a session.

Notes

The data source object creates one or more sessions. From a session, the consumer can do the following:

- Create a command by calling **IDBCreateCommand::CreateCommand**, if the provider supports commands. A single session can support multiple commands.
- Create a rowset by calling **IOpenRowset::OpenRowset**. This is similar to creating a rowset over a single table and is supported by all providers.

Building OLE DB provider applications is beyond the scope of this course. For information about building provider applications, search for "Creating an OLE DB Template Provider" in MSDN Library Visual C++ 6.0.

Notes

218 OLE DB Consumer Templates

The OLE DB Consumer Templates simplify the process of developing consumer applications. These templates provide the following features:

- Support for ATL and MFC developers to use the OLE DB interfaces
- Native C/C++ data types, such as **int** and **char**, for OLE DB programming
- An easy-to-use binding model for database parameters and columns

  For example, with the **CManualAccessor** class, you can use whatever data types you want as long as the provider can convert the type. It handles both result columns and command parameters.
- OLE DB features, such as provider-owned memory for improved performance and multiple accessors on a rowset

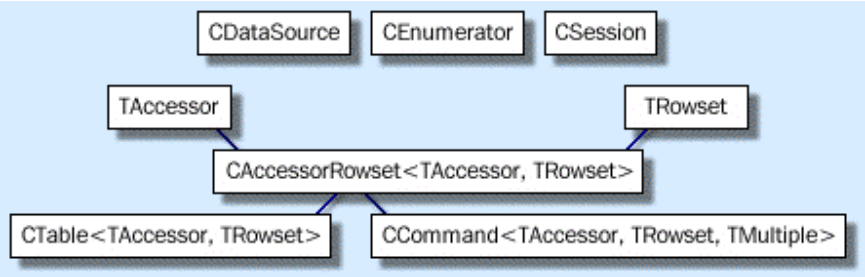
Using these templates, you can create applications that:

- Retrieve records from a data source for further processing.
- Execute parameterized queries to retrieve records selectively.

OLE DB Consumer Architecture

The classes in the top layer of OLE DB Consumer Templates conform to the OLE DB specification as closely as possible. Each class mirrors an existing OLE DB component. For example, the **CDataSource** class corresponds to the data source object in OLE DB.

The following illustration shows the OLE DB consumer classes. To display an overview of a class, click its label. (CD-ROM displays context-sensitive text.)



For more information about consumer classes, search for "OLE DB Consumer Templates Architecture" in MSDN Library Visual Studio 6.0.

Notes

219 Using CAccessor to Create a Consumer Application

You use MFC AppWizard to quickly create a base consumer application that uses **CAccessor**. You can then add code to the base application to include functionality, such as the addition and deletion of records in a data source.

This section describes how to use the **CAccessor** class to create a consumer application. You use the **CAccessor** class when you know at design time the structure and type of database you want to access.

This section includes the following topics:

- Creating a Base Application Using MFC AppWizard
- MFC AppWizard Implementation of CAccessor
- COleDBRecordView Class
- Performing Database Operations

Notes

220 Creating a Base Application Using MFC AppWizard

The first step in building a consumer application with **CAccessor** is to use MFC AppWizard to build a base application.

To see a demonstration that shows how to use MFC AppWizard to build the base application, click this icon.

(CD-ROM plays the demonstration, "Creating an OLEDB Application by Using AppWizard.")

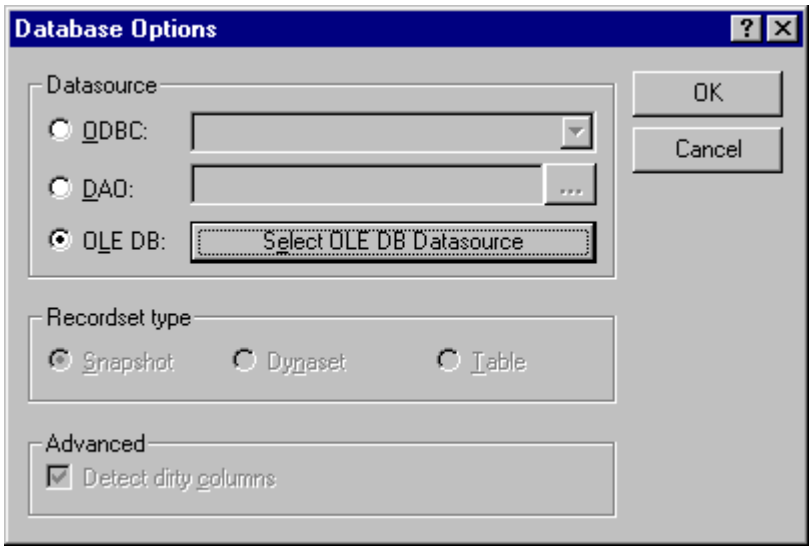
▶ **To create an application by using MFC AppWizard**

1. Invoke MFC AppWizard. Accept the default settings in the **Step 1 of 6** dialog box.
2. In the **Step 2 of 6** dialog box, click **Database view with file support** and then click **Data Source**.

   The **Database Options** dialog box appears.

   To see an illustration of the **Database Options** dialog box, click this icon.

   (CD-ROM displays the following illustration.)

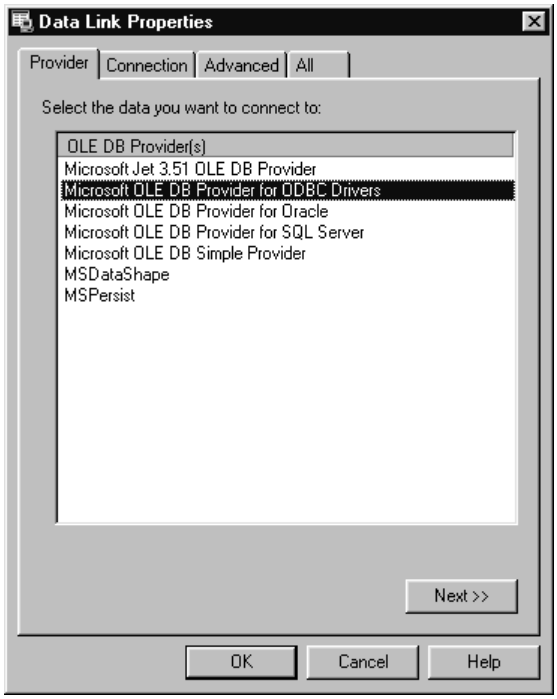


Notes

3. Select **OLE DB** and then click **Select OLE DB Datasource**.

   The **Data Link Properties** dialog box appears.

   To see an illustration of the **Data Link Properties** dialog box with the **Provider** tab selected, click this icon.
   (CD-ROM displays the following illustration.)



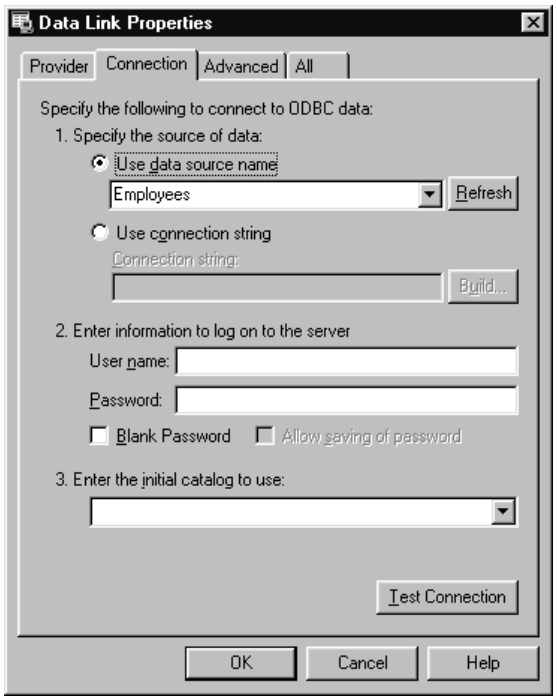
4. Select an OLE DB Provider from the list of providers that are displayed and click **Next**.

Notes

5. On the **Connection** tab, select or enter the location of the data source.

   If you choose a provider such as Microsoft Jet OLE DB Provider, specify the full path name of the database file. If you choose a provider such as Microsoft OLE DB Provider for ODBC Drivers, specify the data source name. If you choose Microsoft OLE DB Provider for Oracle, you must give the Oracle server name as the location.

   To see an illustration of the **Data Link Properties** dialog box with the **Connection** tab selected, click this icon.
    (CD-ROM displays the following illustration.)

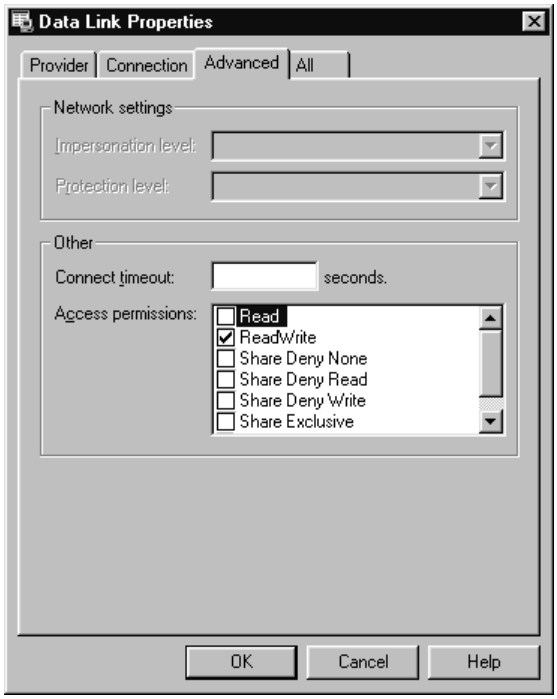


6. Enter the user name and password with which to connect to the data source.
7. Click **Test Connection** to test the connection to the provider.

Notes

8. To set advanced options, click the **Advanced** tab.

   To see an illustration of the **Data Link Properties** dialog box with the **Advanced** tab selected, click this icon.
    (CD-ROM displays the following illustration.)



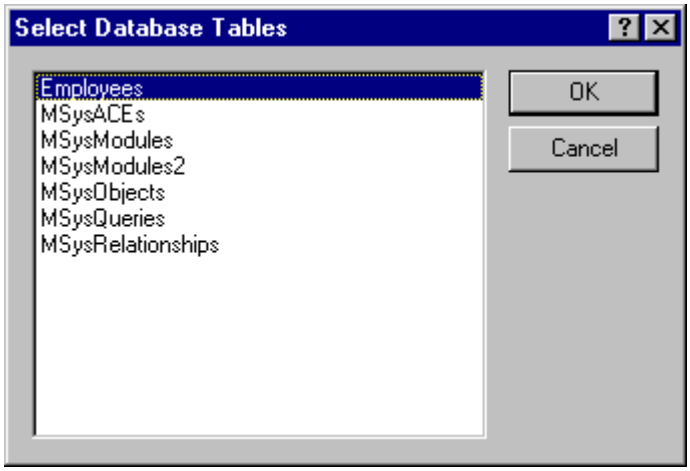
   You can change the setting of any option. For example, in the **Access permissions** list box, you can select the **ReadWrite** check box.

Notes

9 Click **OK** to return to the **Database Options** dialog box and click **OK**.

The **Select Database Tables** dialog box appears. To see an illustration of this dialog box, click this icon.

 (CD-ROM displays the following illustration.)



10. Select the table you want to access and click **OK**.
11. Click **Finish** to create the base application.

Notes

221 MFC AppWizard Implementation of CAccessor

MFC AppWizard uses the classes **CAccessor** and **CCommand** to access data from the data source. The wizard derives a class named **CXXXXSet** from **CCommand**, which inherits the **CAccessor** class. The **CAccessor** class in turn inherits **CEmployees**, which defines the structure of the records, as shown in the following example:

```
class CXXXXSet  :   public CCommand<CAccessor<CEmployees>>
```

To see the class definition of **CEmployees**, click this icon.

 (CD-ROM displays the following sample code.)

```
// Class Definition of CEmployees
// Samples\CH07\OleEmployee
class CEmployees
{
    public:
        int m_EmployeeID;
        char m_Name[51];
        char m_Address[51];


    BEGIN_COLUMN_MAP(CEmployees)
        COLUMN_ENTRY_TYPE(1, DBTYPE_I4, m_EmployeeID)
        COLUMN_ENTRY_TYPE(2, DBTYPE_STR, m_Name)
        COLUMN_ENTRY_TYPE(3, DBTYPE_STR, m_Address)
    END_COLUMN_MAP()

};
```

Note **CCommand** also inherits the **CRowset** class by default.

MFC AppWizard overrides the **Open** function of the **CCommand**-derived class. In the **Open** function, objects of **CDataSource** and **CSession** are defined, as shown in the following example:

```
public:

    HRESULT Open()
    {
        CDataSource db;
        CSession    session;
        ...
    }
```

Note When MFC AppWizard generates the code for your application, the scope of the **CSession** object is limited to the **Open** function of the **CCommand<CAccessor<C*Tablename* >>**-derived class.

Notes

MFC AppWizard defines an object of **CDBPropSet**, as shown in the following example:

```
CDBPropSet  dbinit(DBPROPSET_DBINIT);
```

The **CDBPropSet** object is used to define the properties of the data source object. These properties are needed to open the data source object.

To see sample code that shows the definition of data source properties, click this icon.
 (CD-ROM displays the following sample code.)

```
//Defining data source object properties
// Samples\CH07\OleEmployee
...
HRESULT Open()
{
    ...
    CDBPropSet  dbinit(DBPROPSET_DBINIT);
    dbinit.AddProperty(DBPROP_INIT_DATASOURCE, "Employees");
    dbinit.AddProperty(DBPROP_INIT_MODE, (long)3);
    dbinit.AddProperty(DBPROP_INIT_PROMPT, (short)4);
    dbinit.AddProperty(DBPROP_INIT_LCID, (long)1033);
    ...
}
```

You use the **CDataSource** member function **Open** to open the data source. The first argument of the **Open** function is the name of the data provider, and the second argument is the **CDBPropSet** object, which defines the properties of the data source. The following example opens a Microsoft Jet database by using the Microsoft OLE DB Provider for ODBC:

```
hr = db.Open("MSDASQL.1", &dbinit);
```

You use the **Open** function of the **CSession** class to open a session, as shown in the following example:

```
hr = session.Open(db);
```

After a session object is opened successfully and the properties of the rowset are defined, you can call the **Open** function of the **CCommand** class. The **Open** function takes as arguments the session object, the command text, which is an SQL statement, and the propertyset object.

Notes

To see sample code that sets rowset properties and calls the **Open** function, click this icon.
 (CD-ROM displays the following sample code.)

```
// Setting properties of a rowset and opening it
// Samples\CH07\OleEmployee
...
HRESULT Open()
{
    ...
    CDBPropSet  propset(DBPROPSET_ROWSET);

    //The next 2 lines enable scrolling for the rowset
    propset.AddProperty(DBPROP_CANFETCHBACKWARDS, true);
    propset.AddProperty(DBPROP_IRowsetScroll, true);

    //The next 2 lines allow you to make changes to the rowset contents
    propset.AddProperty(DBPROP_IRowsetChange, true);
    propset.AddProperty(DBPROP_UPDATABILITY, DBPROPVAL_UP_CHANGE);

    hr = CCommand<CAccessor<CEmployees> >::Open(session,
         "SELECT * FROM Employees", &propset);
    ...
}
```

Notes

224 COleDBRecordView Class

When you create an OLE DB consumer application by using MFC AppWizard, the view class is derived from the **COleDBRecordView** class by default. **COleDBRecordView** displays database records in controls. **COleDBRecordView** is derived from **CFormView** and therefore is created using a dialog template. The view displays the fields of the **CRowset** object in the dialog box controls. You can implement functions that add and delete records and implement transaction processing.

A pointer to the **CCommand<CAccessor<C*Tablename*> >** class is declared within the definition of the view class, as shown in the following example:

```
class CXXXXView : public COleDBRecordView
{
    ...
public:
    CXXXXSet* m_pSet;
    ...
}
```

In the **OnInitialUpdate** function of the view class, use the **Open** function to open the recordset, as shown in the following example:

```
m_pSet->Open();
```

To see sample code that shows the complete definition of **OnInitialUpdate**, click this icon.
 (CD-ROM displays the following sample code.)

```
// Samples\CH07\OleEmployee
//The OnInitialUpdate function

void CXXXXView::OnInitialUpdate()
{
    m_pSet = &GetDocument()->m_oleEmployeeSet;
    {
        CWaitCursor wait;
        HRESULT hr = m_pSet->Open();
        if (hr != S_OK)
        {
            AfxMessageBox(_T("Record set failed to open."), MB_OK);
            // Disable the Next and Previous record commands,
            // since attempting to change the current record without an
            // open RecordSet will cause a crash.
            m_bOnFirstRecord = TRUE;
            m_bOnLastRecord = TRUE;
        }
    }
    COleDBRecordView::OnInitialUpdate();

}
```

Notes

In the previous example, m_bOnFirstRecord and m_bOnLastRecord are member variables of the **COleDBRecordView** class. Setting these variables to **TRUE** ensures that the navigation buttons on the toolbar and their corresponding menu command items are disabled.

Notes

225 Performing Database Operations

To display the contents of a recordset or to perform operations such as adding and deleting records, you need to add code to your application.

Displaying the Recordset

To display the contents of the recordset, complete the following steps:

1. Place controls on the dialog box created by MFC AppWizard.
2. Define member variables for the controls using ClassWizard.

   ClassWizard adds suitable DDX functions to handle the transfer of data from the controls to the member variables.
3. Alter the DDX functions to transfer data between the controls and the recordset variables, as shown in the following example:

   ```
   //{{AFX_DATA_MAP(COLEEmployeeView)
   //}}AFX_DATA_MAP
   DDX_Text(pDX, IDC_ADDRESS, m_pSet->m_Address,51);
   DDX_Text(pDX, IDC_EMPLOYEEID, m_pSet->m_ID);
   DDX_Text(pDX, IDC_NAME, m_pSet->m_Name,51);
   ```

   In the previous example, the DDX functions have been placed outside AFX_DATA_MAP. This is because the example alters the DDX functions to use the rowset's member variables instead of those defined by ClassWizard. If the DDX functions were placed within AFX_DATA_MAP, the functionality of ClassWizard for the **COleDBRecordView**-derived class would be impaired.

The view has built-in functionality to scroll through the records in the recordset. You can also write your own handlers for the menu commands ID_RECORD_FIRST, ID_RECORD_NEXT, ID_RECORD_PREV and ID_RECORD_LAST generated by MFC AppWizard.

The following example implements a handler for ID_RECORD_LAST:

```
void CXXXXView::OnRecordLast()
{
    m_pSet->Update();
    m_pSet->MoveLast();
    UpdateData(FALSE);
}
```

You invoke the **Update** member function of **CRecordset** to update the database with any changes made to the current record. If you call **UpdateData** and pass **FALSE** as the argument, the controls on the form are updated with the data of the current record.

Notes

Adding Records to a Data Source

You add records to a data source by using the **Insert** member function of **CRecordset**. Before invoking the **Insert** function, you must invoke the **UpdateData** function, as shown in the following example:

```
void CXXXXView::OnRecordAdd()
{
    UpdateData(TRUE);
    m_pSet->Insert();
    .....
}
```

When you call **UpdateData** and pass **TRUE** as the argument, the current record is updated with the data of the controls.

Before you open the accessor, you must set the DBPROP_UPDATABILITY property to allow inserts, as shown in the following example:

```
propset.AddProperty(DBPROP_UPDATABILITY,
BPROPVAL_UP_CHANGE|DBPROPVAL_UP_INSERT|DBPROPVAL_UP_DELETE);
hr = CCommand<CAccessor<CEmployees> >::Open(session,
"SELECT * FROM Employees", &propset);
.....
```

In the previous example, DBPROP_UPDATABILITY has also been set to allow deletes and edits.

Deleting Records from a Data Source

You can delete records from a data source using the **Delete** member function of **CRecordSet**. This function deletes the record currently pointed to by the recordset pointer. After invoking the **Delete** function, you must invoke the **OnMove** function passing ID_RECORD_NEXT as the argument, to set the recordset pointer to the record next to the deleted record. You then invoke **UpdateData** to display the current record in the form.

The following example shows how to delete the current record:

```
void COLEEmployeeView::OnRecordDelete()
{
    m_pSet->Delete();
    if(!OnMove(ID_RECORD_NEXT))
    {
        if(!OnMove(ID_RECORD_LAST))
        {
            m_pSet->m_EmployeeID = 0;
            m_pSet->m_Name[0] = '\0';
            m_pSet->m_Address[0]= '\0';
            m_bOnFirstRecord = TRUE;
            m_bOnLastRecord = TRUE;
        }
    }
    UpdateData(FALSE);
}
```

Notes

Editing Records in a Data Source

You do not need to write code to update existing records. You can simply edit the contents of the controls and scroll to the next or previous record. The **OnMove** function is called when you scroll to another record and ensures that the recordset is updated before the recordset pointer is moved.

Handling Transactions

OLE DB applications use the **CSession** object to implement transaction processing. The **CSession** object uses the ITransaction interface to commit, abort, and obtain status information about transactions. You use the **StartTransaction** member function of the **CSession** class to begin a transaction, as shown in the following example:

```
session.StartTransaction();
```

Changes are not written back to the data source until you invoke the **Commit** member function, as shown in the following example:

```
m_pSet->session.Commit()
```

Notes

231 Using CDynamicAccessor to Create a Consumer Application

You can use the **CDynamicAccessor** class to retrieve data from a data source when you do not know the structure of the data source at design time. This class creates an accessor at run time based on the column information of the rowset. It also creates and manages a buffer for the data.

If you used MFC AppWizard to create a **CAccessor**-based application, you can modify the application to use **CDynamicAccessor**.

▶ **To modify a CAccessor-based application to use CDynamicAccessor**

1. Delete the XXXXSet.h and XXXXSet.cpp files from the project.

   These files define the recordset for the **CAccessor** class. Because the **CDynamicAccessor** class maintains its own buffer, these files are no longer required.
2. Delete all references to the XXXXSet.h file from all files in the project.
3. Delete all references to the CXXXXSet class from all files in the project.
4. Delete the **OnGetRowSet** function from the **CXXXXView** class.
5. Delete all references to the pointer m_pSet.
6. In the XXXXView.h and XXXXView.cpp files, replace all occurrences of **COleDBRecordView** with **CFormView**.

   MFC AppWizard derives the **CXXXXView** class from the **COleDBRecordView** class. **COleDBRecordView** is a form view that is directly connected to a **CRowset** object. The **COleDBRecordView** object uses the **CRowset** object to provide a default implementation for moving to the first, next, previous, or last record. However, because you are not making use of the scrolling functionality provided by the **COleDBRecordView**, you will derive the view from the **CFormView** class.
7. In the **CXXXXView** class, declare protected member variables of the **CDataSource** class, the **CSession** class, and the **CCommand<CDynamicAccessor>** class, as shown in the following example:

   ```
   CDataSource m_DataSource;
   CSession m_Session;
   CCommand<CDynamicAccessor> DynamicAccessor;
   ```
8. Declare a protected member variable of type **integer** and an array of void pointers, as shown in the following example:

   ```
   int nColCnt;
   void *pVoid[3];
   ```

   The integer variable is used to store the count of the number of columns in the table being accessed. The array of void pointers is used to access the data in the buffer defined by the **CDynamicAccessor** class. In the previous example, the size of the array is defined as 3. However, you can set the size at run time by using the **new** operator and the number of columns in the table.

Notes

9. For each field in the table, create a control on the **FormView** dialog box and define suitable member variables for each control, as shown in the following example:

```
long    m_EmployeeID;
char    m_Address[51];
char    m_Name[51];
```

Note In the previous example, the controls have been created at design time. You can create them at run time using each control's **Create** member function.

10. In the **OnInitialUpdate** function of the view class, create an object of type **CDBPropSet** and use the **AddProperty** member function to add properties to the property set.

To see sample code that sets the properties of the property set, click this icon:
 (CD-ROM displays the following sample code.)

```
//Samples\Ch07\DynAcc01
//Adding properties to the CDBPropSet object

CDBPropSet  dbinit(DBPROPSET_DBINIT);
dbinit.AddProperty(DBPROP_AUTH_CACHE_AUTHINFO, true);
dbinit.AddProperty(DBPROP_AUTH_ENCRYPT_PASSWORD, false);
dbinit.AddProperty(DBPROP_AUTH_MASK_PASSWORD, false);
dbinit.AddProperty(DBPROP_AUTH_PASSWORD, "");
dbinit.AddProperty(DBPROP_AUTH_PERSIST_ENCRYPTED, false);
dbinit.AddProperty(DBPROP_AUTH_PERSIST_SENSITIVE_AUTHINFO, false);
dbinit.AddProperty(DBPROP_AUTH_USERID, "Admin");
dbinit.AddProperty(DBPROP_INIT_DATASOURCE, "d:\\databases\\emp.mdb");
dbinit.AddProperty(DBPROP_INIT_MODE, (long)3);
dbinit.AddProperty(DBPROP_INIT_PROMPT, (short)2);
dbinit.AddProperty(DBPROP_INIT_PROVIDERSTRING,
";COUNTRY=0;CP=1252;LANGID=0x0409");
dbinit.AddProperty(DBPROP_INIT_LCID, (long)1033);
```

Note In the previous example, the DBPROP_INIT_DATASOURCE property has been set to a specific data source. You can set this property to a data source at run time by allowing the user to choose from a list of data sources. For an example of how to implement this functionality, see the sample project DynAcc02 in Samples\Ch07.

11. Open the data source object by using the **Open** function, as shown in the following example:

```
m_DataSource.Open("Microsoft.Jet.OLEDB.3.51", &dbinit);
```

Note In the previous example, the name of the provider, Microsoft.Jet.OLEDB.3.51, was passed to the **Open** function. You can display a list of providers and allow the user to select from this list at run time. For an example of how to implement this functionality, see the sample project DynAcc02 in Samples\Ch07.

Notes

12. Using the **Open** function, open a session on the data source, as shown in the following example:

```
m_session.Open(m_DataSource);
```

13. Set properties for the accessor by using a **CDBPropSet** object, as shown in the following example:

```
CDBPropSet  propset(DBPROPSET_ROWSET);
propset.AddProperty(DBPROP_CANFETCHBACKWARDS, true);
propset.AddProperty(DBPROP_IRowsetChange, true);
propset.AddProperty(DBPROP_UPDATABILITY, DBPROPVAL_UP_CHANGE);
```

14. Invoke the **CCommand<CDynamicAccessor>** member function **Open** to access the records in the data source table, as shown in the following example:

```
DynamicAccessor.Open(m_session,"Select * from employees",&propset);
```

Note The previous example sets the SQL statement to be used to open the accessor at design time. You can set the SQL statement at run time. You can display a list of tables available in the data source and allow the user to select the table to be opened. You can then read the schema of the table and display a list of columns for the user to choose from. Using the columns selected, you can generate an SQL statement and use it with the **Open** function to open the accessor. For an example of how to implement this functionality, see the sample project DynAcc02 in Samples\Ch07.

You can use the **CRowset** member functions **MoveFirst**, **MoveNext**, **MoveLast**, and **MovePrev** to scroll through the records in the recordset. The following table lists other functions that provide information about the data in the dynamic accessor's buffer.

Function	Description
GetColumnCount	Returns a count of the number of columns in the data source table.
GetColumnType	Returns the data type of a specific column in the data source table. For example, the code: `DynamicAccessor.GetColumnType(1, &mType);` stores the data type of column 1 in the variable mType, which is of type **DBTYPE**.
GetValue	Returns the data from a specific column in the current record. For example, the code: `pVoid = DynamicAccessor.GetValue(1);` returns a pointer to the value stored in column one. Column numbers start from 1. The pointer pVoid is a void pointer that can point to any data type.

Notes

237 Using CManualAccessor to Create a Consumer Application

You use the **CManualAccessor** class when:

- The structure of the data source is not known, and
- You want to define custom binding specifications at run time.

CManualAccessor enables you to specify the binding information, whereas **CDynamicAccessor** generates the binding information for you.

If you used MFC AppWizard to create a **CAccessor**-based application, you can modify the application to use **CManualAccessor**. You need to delete the XXXXSet.h and XXXXSet.cpp files from the project and delete all references to the XXXXSet.h file. Delete all references to the class CXXXXSet and the pointer m_pSet across all project files. Delete the **OnGetRowSet** function from the **CXXXXView** class. In the XXXXView.h and XXXXView.cpp files, replace all occurrences of **COleDBRecordView** with **CFormView**.

To use a manual accessor to access records in a data source, you need to do the following:

1. Open the root enumerator and traverse the rowset describing the enumerators and data sources available.

   Enumerators have attributes m_szDescription, m_szName and m_nType. The attribute m_szDescription provides a short description of the enumerator, m_szName contains the name of the enumerator or data source, and m_nType stores the type of enumerator. If m_nType has a value of 1, then the enumerator is a data source and can be used directly. If m_nType has a value of 2, then it is an enumerator that is associated with underlying data sources. You open the root enumerator by using the **Open** function as shown in the following example:

   ```
   m_RootEnum.Open(&CLSID_OLEDB_ENUMERATOR);
   ```

   In the previous example, **m_RootEnum** is an object of the **CEnumerator** class. The **CEnumerator** class exposes the ISourcesRowset interface, which returns a rowset describing all data sources and enumerators.

   You can traverse the enumerators available with the root enumerator by using the **MoveNext**, **MoveFirst**, **MovePrev** and **MoveLast** functions, as shown in the following example:

   ```
   if(m_RootEnum.MoveFirst() == S_OK)
   {
   ...
       do
       {
       .....
       }while (m_RootEnum.MoveNext() == S_OK);
       .....
   }
   ```

Notes

To see sample code that traverses the rowset of enumerators and displays their names in a list box, click this icon.

 (CD-ROM displays the following sample code.)

```
//Samples\Ch07\ManAcc01
//Traverses the list of enumerators and displays their names in a
// listbox

BOOL CEnumDlg::OnInitDialog()
{
    USES_CONVERSION;
    CDialog::OnInitDialog();

    m_plbEnum=(CListBox*)GetDlgItem(IDC_ENUMERATOR);
    if(m_RootEnum.m_spSourcesRowset==NULL)
    {
        m_pOK=(CButton*)GetDlgItem(IDOK);
        m_pOK->EnableWindow(FALSE);
        if (m_RootEnum.Open(&CLSID_OLEDB_ENUMERATOR) != S_OK)
        {
            ::AfxMessageBox("Failed to Open Enumerator");
            OnCancel();
        }
        else
        {
            m_pOK->EnableWindow(TRUE);
        }
    }
    if(m_RootEnum.MoveFirst() == S_OK)
    {
        int i=0;
        do
        {
            m_plbEnum->AddString(W2T(m_RootEnum.m_szDescription));
            m_listEnum[i]=W2T(m_RootEnum.m_szName);
            i++;
        }while (m_RootEnum.MoveNext() == S_OK);
        m_plbEnum->SetCurSel(0);
    }
    return TRUE;
}
```

Notes

To open an underlying data source for m_nType 2 root enumerators, complete the following steps:

- Declare a COM pointer, as shown in the following example:

  ```
  CComPtr<IMoniker> spMoniker;
  ```

- Use the pointer to get a moniker on the root enumerator, as shown in the following example:

  ```
  m_pRootEnum->GetMoniker(&spMoniker, W2T(m_pRootEnum->m_szParseName));
  ```

- Use the moniker to gain access to the underlying data sources, as shown in the following example:

  ```
  Enum.Open(spMoniker);
  ```

  In the previous example, **Enum** is an object of **CEnumerator** type.

- Traverse the data sources available by using the **MoveFirst**, **MovePrev**, **MoveNext**, and **MoveLast** functions, as shown in the following example:

  ```
  if(m_Enum.MoveFirst()==S_OK)
  {
      do
      {
          ......
      }while (m_Enum.MoveNext() == S_OK);
      .......
  }
  ```

Notes

To see sample code that displays a list of data sources in a dialog box, click this icon.
 (CD-ROM displays the following sample code.)

```
//Samples\Ch07\ManAcc01
//Gets a moniker onthe root enumerator and accesses the underlying data
//sources. The function displays a list of available data sources.
BOOL CDataSourceDlg::OnInitDialog()
{
    USES_CONVERSION;

    CDialog::OnInitDialog();
    m_pDataSourcePath=(CEdit*)GetDlgItem(IDC_DATASOURCE_PATH);
    m_pDataSource=(CComboBox*)GetDlgItem(IDC_DATASOURCE_NAME);
    m_pOK=(CButton*)GetDlgItem(IDOK);

    if(m_pRootEnum->m_nType==1)
    {
        m_pDataSourcePath->EnableWindow(TRUE);
        m_pDataSource->EnableWindow(FALSE);
    }
    else
    {
        m_pDataSourcePath->EnableWindow(FALSE);
        m_pDataSource->EnableWindow(TRUE);

        CComPtr<IMoniker> spMoniker;
```

(Code continues on following page.)

Notes

```
        if(m_pRootEnum->GetMoniker(&spMoniker, W2T(m_pRootEnum-
>m_szParseName))==S_OK)
        {
            if(m_Enum.m_spSourcesRowset==NULL)
            {

                if (m_Enum.Open(spMoniker) != S_OK)
                {
                    ::AfxMessageBox("Failed to Open Data Source");
                    m_pOK->EnableWindow(FALSE);
                    OnCancel();
                    return FALSE;
                }
                else
                {
                    m_pOK->EnableWindow(TRUE);
                }
            }

            int i=0;
            if(m_Enum.MoveFirst()==S_OK)
            {
                do
                {
                    m_pDataSource->AddString(W2T(m_Enum.m_szName));
                    m_listDataSource[i]= W2T(m_Enum.m_szName);
                    i++;
                }while (m_Enum.MoveNext() == S_OK);
                m_pDataSource->SetCurSel(0);
            }
            else
            {
                m_pOK->EnableWindow(FALSE);
                OnCancel();
                return FALSE;
            }

        }

    }
    return TRUE;
}
```

Notes

2. Open the data source by using the **Open** function, as shown in the following example:

   ```
   m_source.Open(*m_pRootEnum,&dbinit)
   ```

   The previous example opens the data source by using the root enumerator. The **m_source** object is of type **CDataSource**, and **dbinit** is of type **CDBPropSet**. You use the **AddProperty** function of the **dbinit** object to define properties for the data source. The following example sets the **USERID** property to **Admin**:

   ```
   dbinit.AddProperty(DBPROP_AUTH_USERID, "Admin");
   ```

   To see sample code that sets properties and opens a data source, click this icon.
    (CD-ROM displays the following sample code.)

   ```
   //Samples\Ch07\ManAcc01
   //Sets properties and opens a data source
   void CDataSourceDlg::OnKillfocusDatasourcePath()
   {
       USES_CONVERSION;
       UpdateData(TRUE);
       CDBPropSet  dbinit(DBPROPSET_DBINIT);
       dbinit.AddProperty(DBPROP_AUTH_CACHE_AUTHINFO, true);
       dbinit.AddProperty(DBPROP_AUTH_ENCRYPT_PASSWORD, false);
       dbinit.AddProperty(DBPROP_AUTH_MASK_PASSWORD, false);
       dbinit.AddProperty(DBPROP_AUTH_PASSWORD, "");
       dbinit.AddProperty(DBPROP_AUTH_PERSIST_ENCRYPTED, false);
       dbinit.AddProperty(DBPROP_AUTH_PERSIST_SENSITIVE_AUTHINFO, false);
       dbinit.AddProperty(DBPROP_AUTH_USERID, "Admin");
       dbinit.AddProperty(DBPROP_INIT_DATASOURCE, m_Path);
       dbinit.AddProperty(DBPROP_INIT_MODE, (long)16);
       dbinit.AddProperty(DBPROP_INIT_PROMPT, (short)2);
       dbinit.AddProperty(DBPROP_INIT_PROVIDERSTRING,
   ";COUNTRY=0;CP=1252;LANGID=0x0409");
       dbinit.AddProperty(DBPROP_INIT_LCID, (long)1033);
       if (m_source.Open(*m_RootEnum,&dbinit)!= S_OK)
       {
           ::AfxMessageBox("Failed to Open Data Source");
           m_pOK->EnableWindow(FALSE);
           OnCancel();
           return;
       }
   }
   ```

3. Open a session on the data source by using a **CSession** object, as shown in the following example:

   ```
   m_pSession->Open(*m_pSource);
   ```

Notes

4. Open the tables within the data source by using a **CTables** object, as shown in the following example:

   ```
   m_Tbl.Open(*m_pSession,NULL,NULL,NULL,"TABLE")
   ```

   In the previous example, **m_Tbl** is a **CTables**-type object. The **Open** function takes five arguments; the first argument is the session object, and the remaining arguments are restrictions that can be applied when the **CTables** object is opened. The last argument to the **Open** function ensures that only user-defined tables are accessed.

   You can traverse the tables available in the data source by using the **MoveFirst**, **MovePrev**, **MoveNext**, and **MoveLast** functions, as shown in the following example:

   ```
   if(m_Tbl.MoveFirst()==S_OK)
   {
       .....
       do
       {
           ....
       }while (m_Tbl.MoveNext() == S_OK);
       ....
   }
   ```

Notes

5. Determine the schema of the table by using a **CColumns** object, as shown in the following example:

```
m_Col.Open(*m_pSession,NULL,NULL,m_Tbl->m_szName);
```

The object **m_Col** is of type **CColumns**. The last argument to the **Open** function is an attribute of the **CTables** object, which contains the name of the table whose column information is to be accessed. The schema can be used to construct the SQL statement that is required to retrieve that data from the data source. To see sample code that constructs an SQL statement using the fields selected by the user, click this icon.

 (CD-ROM displays the following sample code.)

```
//Samples\Ch07\ManAcc01
//Generates an SQL statement form the list of columns the user selects

void CColumnsDlg::OnKillFocus(CWnd* pNewWnd)
{
    int nLbSelItems[100];
    CDialog::OnKillFocus(pNewWnd);

    m_plbColumns->GetSelItems(m_fld_cnt,nLbSelItems);
    m_ColsSelected=m_plbColumns->GetSelCount();
    m_cmd_str="Select";
    for(int j = 0 ; j < m_ColsSelected; j++)
    {
        if(j==0)
        {
            m_cmd_str=m_cmd_str+ "  " + m_listColumns[nLbSelItems[j]];
        }
        else
        {
            m_cmd_str=m_cmd_str+ " , " + m_listColumns[nLbSelItems[j]];
        }

    }
    if(m_ColsSelected==0)
    {
        m_ColsSelected=m_fld_cnt;
        m_cmd_str="Select * ";
    }
    m_cmd_str=m_cmd_str + " FROM ";
    m_cmd_str=m_cmd_str + m_pTbl->m_szName;

}
```

6. Declare an object of type **CCommand<CManualAccessor>**, as shown in the following example:

```
CCommand<CManualAccessor> manAcc;
```

Notes

7. Invoke the **CreateAccessor** function to create the manual accessor, as shown in the following example:

   ```
   manAcc.CreateAccessor(m_ColsSelected, pBind, sizeof(MyBind)*m_ColsSelected);
   ```

   In the previous example, the first argument defines the number of columns to be accessed, the second argument is a pointer to the buffer where the data that is retrieved is to be stored, and the third argument defines the size of the buffer. To see sample code that defines the buffer, click this icon.  (CD-ROM displays the following sample code.)

   ```
   //Samples\Ch07\ManAcc01
   //Defines a buffer for the manual accessor

   struct MyBind
   {
       MyBind()
       {
           memset(this, 0, sizeof(*this));
       }

       TCHAR   szValue[40];
       DWORD   dwStatus;
   };
   ```

8. Open the manual accessor by using the **Open** function, as shown in the following example:

   ```
   manAcc.Open(m_session,m_cmd_str);
   ```

   In the previous example, **m_session** is the session that was opened earlier on the data source, and **m_cmd_str** contains the SQL statement to be used to access the data.

9. Traverse the records in the table by using the **MoveFirst**, **MovePrev**, **MoveNext**, and **MoveLast** functions.

Notes

243 Sample Applications

The following table describes the sample applications related to this chapter. These sample applications are located in the folder \Samples\Ch07.

Sample application subfolder	Description of application
OLEEmployee	An OLE DB Templates-based application that makes use of the **CAccessor** class to access data from a specific data source using a specific provider. Before you run this application, you need to create an ODBC data source called Employees, by using the ODBC Data Source Administrator. Ensure that the database file has read-write permissions.
DynAcc01	An OLE DB Templates-based application that makes use of the **CDynamicAccessor** class to access data from a specific data source using a specific provider. Ensure that the database file has read-write permissions.
DynAcc02	An OLE DB Templates-based application that makes use of the **CDynamicAccessor** class to access data from any data source. The user can choose from a list of providers, select a data source, select a table, select columns from the table, and then display that data stored in the table. When prompted to specify the path to the data source, in the **Data Source** dialog box, specify the complete path to the data source. Ensure that the data source has read-write permissions.
ManAcc01	An OLE DB Templates-based application that makes use of the **CManualAccessor** class to access data from any data source. The user can choose from a list of providers, select a data source, select a table, select columns from the table, and then display that data stored in the table. When prompted to specify the path to the data source, in the **Data Source** dialog box, specify the complete path to the data source. Ensure that the data source has read-write permissions.

Notes

Self-Check Questions

1. What is the role of a data provider?

- A. Expose the data in a tabular format.
- B. Facilitate data access by a consumer.
- C. Retrieve data from a data source.
- D. Store data in a tabular format.

2. Which OLE DB component helps you search for data sources?

- A. Data source objects
- B. Sessions
- C. Commands
- D. Enumerators

3. Which base class of the consumer template classes encapsulates a rowset and its accessors?

- A. **CTable**
- B. **CCommand**
- C. **CAccessorRowset**
- D. **CDataSource**

4. Which function creates a data source object?

- A. **OpenRowset**
- B. **CoCreateInstance**
- C. **Initialize**
- D. **CreateSession**

5. Which is the default base view class for consumer applications that use CAccessor?

- A. **CFormView**
- B. **CView**
- C. **COleDBRecordView**
- D. **CListView**

Notes